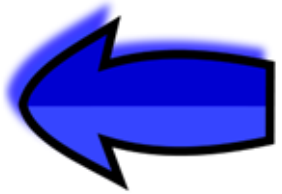
# Chapter 2:  The Language of Bits

# Outline

* Boolean Algebra 
* Positive Integers
* Negative Integers
* Floating-Point Numbers
* Strings

# What does a Computer Understand ?

* Computers do not understand natural human languages, nor programming languages
* They only understand the language of bits

| | |
|---|---|
| Bit | 0 or 1 |
| Byte | 8 bits |
| Word | 4 bytes |
| kiloByte | 1024 bytes |
| megaByte | $10^6$ bytes |



3

# Review of Logical Operations

* A + B (A or B)

| A | B | A + B |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 1 | 1 |

* A.B ( A and B)

| A | B | A.B |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 1 | 1 |

# Review of Logical Operations - II

| A | B | A NAND B |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 1 | 0 |

| A | B | A NOR B |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 1 | 0 |

* NAND and NOR operations
* These are universal operations. They can be used to implement any Boolean function.

# Review of Logical Operations

* XOR Operation : $(A \oplus B)$

| A | B | A XOR B |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 1 | 0 |

How many truth tables we can build?

# Review of Logical Operations

* NOT operator
  * Definition: $\overline{0} = 1$, and $\overline{1} = 0$
    * Double negation: $\overline{\overline{A}} = A$, NOT of (NOT of A) is equal to A itself
* OR and AND operators
  * Identity: $A + 0 = A$, and $A.1 = A$
  * Annulment: $A + 1 = 1$, $A.0 = 0$

* Idempotence: $A + A = A$, $A.A = A$, The result of computing the OR and AND of A with itself is A.
* Complementarity: $A + \overline{A} = 1$, $A.\overline{A} = 0$
* Commutativity: $A + B = B + A$, $A.B = B.A$, the order of Boolean variables does not matter
* Associativity: $A+(B+C) = (A+B)+C$, $A.(B.C) = (A.B).C$, similar to addition and multiplication.
* Distributivity: $A.(B + C) = A.B + A.C$, $A+ (B.C) = (A+B).(A+C)$ → Use this law to open up parantheses and simplify expressions

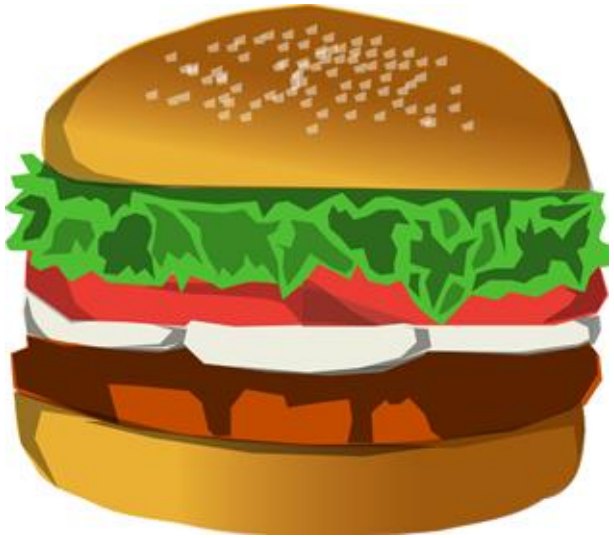
# De Morgan's Laws

* Two very useful rules

$$\overline{A + B} = \overline{A}.\overline{B}$$

$$\overline{A.B} = \overline{A} + \overline{B}$$

# Consensus Theorem



$XY + \bar{X}Z + YZ(X + \bar{X})$

$= XY + \bar{X}Z + XYZ + Y\bar{X}Z$

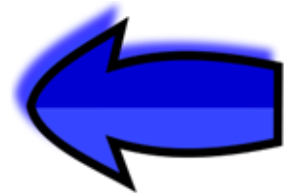
$= XY(1 + Z) + \bar{X}Z(1 + Y)$

$= XY(1) + \bar{X}Z(1)$

* Prove :
  * $X.Y + \bar{X}.Z + Y.Z = X.Y + \bar{X}.Z$

# Outline

* Boolean Algebra
* Positive Integers
* Negative Integers
* Floating Point Numbers
* Strings



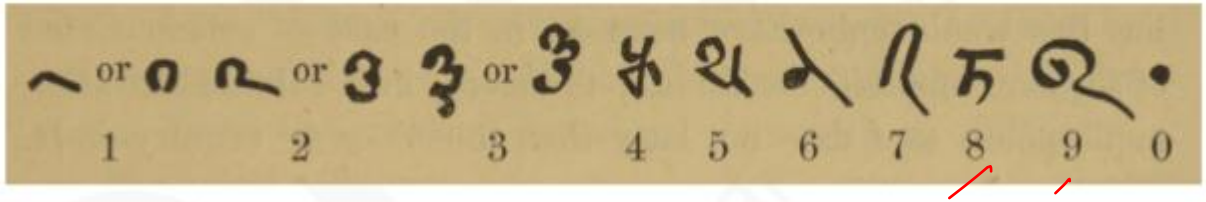
# Representing Positive Integers

* Ancient Roman System

| Symbol | I | V | X | L | C | D | M |
|---|---|---|---|---|---|---|---|
| Value | 1 | 5 | 10 | 50 | 100 | 500 | 1000 |

* Issues :
  * There was no notion of 0
  * Very difficult to represent large numbers
  * Addition, and subtraction (very difficult)

# Indian System (place –value system)




Bakshali numerals, 7th century AD

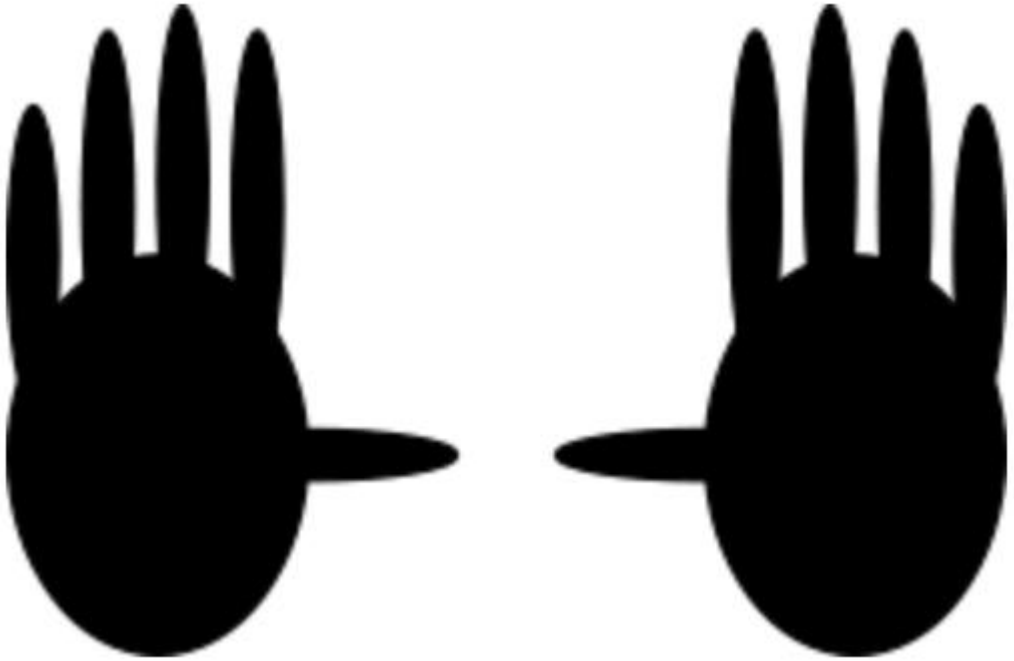
* Uses the place value system

$5301 = 5 * 10^3 + 3 * 10^2 + 0 * 10^1 + 1*10^0$

Example in base 10

# Number Systems in Other Bases

* Why do we use base 10 ?
  * because …



# What if we had a world in which …

* People had only two fingers.

# Binary Number System

* They would use a number system with base 2.

| Number in decimal | Number in binary |
|---|---|
| 5 | 101 |
| 100 | 1100100 |
| 500 | 111110100 |
| 1024 | 10000000000 |

# MSB and LSB

* MSB (Most Significant Bit) → The leftmost bit of a binary number. E.g., MSB of 1110 is 1
* LSB (Least Significant Bit) → The rightmost bit of a binary number. E.g., LSB of 1110 is 0

# Hexadecimal and Octal Numbers

* Hexadecimal numbers
  * Base 16 numbers – 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F
  * Start with 0x
* Octal Numbers
  * Base 8 numbers – 0,1,2,3,4,5,6,7
  * Start with 0

$a_k\ a_{k-1}\ a_{k-2}\ \cdots\ a_0$

$= a_0 \times 2^0 + a_1 \times 2^1 + a_2 \times 2^2 + a_3 2^3 + a_4 2^4 + a_5 2^5 + \cdots$

$= (a_0 2^0 + a_1 2^1 + a_2 2^2) + 2^3 (a_3 2^0 + a_4 2^1 + a_5 2^2) + 2^6 (\quad)$

$b_0 \qquad\qquad b_1$

$= b_0 8^0 + b_1 8^1 + b_2 8^2 + \cdots$

# Examples

Convert 110010111 to the octal format : $\underbrace{110}\,\underbrace{010}\,\underbrace{111}$ = 0627

Convert 111000101111 to the hex format : $\underbrace{1110}\,\underbrace{0010}\,\underbrace{1111}$ = 0xE2F